Galgotias College of Engineering and Technology

Knowledge Park-II, Greater Noida, Uttar Pradesh, 201310



Master of Computer Applications

Software Engineering Lab File - BMC 352

SESSION: 2026-2027 (odd)

SEMESTER - III

Submitted by:

Name: Ankit Chaudhary

Roll No.: 2500970140022

Branch: MCA

Section: A1

Submitted to:

Shavnam Sharma
Assistant Professor of GCET

Sweeti Singh Gautam
Assistant Professor of GCET

Department of Computer Applications

INDEX

S. No	Objective of the Experiment	Date of Experiment	Date of Submission	Sign
1	To prepare a Software Requirements Specification (SRS) document in accordance with IEEE recommended standards.			
2	Draw the use case diagram and specify the role of each of the actors.			
3	Prepare the precondition, postcondition and function of each use case.			
4	Draw the activity diagram.			
5	Identify the classes. Classify them as weak and strong classes and draw the class diagram.			
6	Draw the sequence diagram for any two scenarios.			
7	Draw the collaboration diagram.			
8	Draw the state chart diagram.			
9	Draw the component diagram.			
10	Draw the deployment diagram.			
11	Test case for Video Conferencing System functionality.			
12	Understanding and drawing an ER diagram.			

Experiment No. 1

Aim: To prepare a Software Requirements Specification (SRS) document in accordance with IEEE recommended standards.

Learning Outcomes:

- Students will understand IEEE standard for SRS preparation.
- Learn to define functional and non-functional requirements clearly.
- Gain experience in writing well-structured requirement documents for real-world projects.

Apparatus Used:

- Computer system
- Word processor (e.g., MS Word, Draw.io)
- IEEE SRS standard template

Procedure:

1. Identify the software system to be developed.
2. Define purpose, scope, and intended users.
3. Document system requirements as per IEEE structure.
4. Validate and review the document.

Observation Table:

Software Requirements Specification for Real-Time Video Conferencing System (RTVCS), Version 1.0 – Approved. Prepared by: Amrendra Pal.

Table of Contents

1. Introduction
    1.1 Purpose
    1.2 Document Conventions
    1.3 Intended Audience and Reading Suggestions
    1.4 Product Scope
    1.5 References
2. Overall Description
    2.1 Product Perspective
    2.2 Product Functions
    2.3 User Classes and Characteristics
    2.4 Operating Environment
    2.5 Design and Implementation Constraints
    2.6 User Documentation
    2.7 Assumptions and Dependencies
3. External Interface Requirements
    3.1 User Interfaces
    3.2 Hardware Interfaces
    3.3 Software Interfaces
    3.4 Communications Interfaces
4. System Features
    4.1 System Feature 1 – Create Meeting Room
        4.1.1 Description and Priority
        4.1.2 Stimulus/Response Sequences
        4.1.3 Functional Requirements
    4.2 System Feature 2 – Join Meeting Room
    4.3 System Feature 3 – Audio/Video Control
    4.4 System Feature 4 – Text Chat
    4.5 System Feature 5 – Screen Sharing

4.6 System Feature 6 – Host Controls
5. Other Nonfunctional Requirements
5.1 Performance Requirements
5.2 Safety Requirements
5.3 Security Requirements
5.4 Software Quality Attributes
5.5 Business Rules
6. Other Requirements
Appendix A: Glossary
Appendix B: Analysis Models
Appendix C: To Be Determined List

1. Introduction

1.1 Purpose

This SRS defines the requirements of the Real-Time Video Conferencing System (RTVCS), which lets users hold online meetings using live video, audio, text chat and screen sharing. It is a reference for developers, testers and stakeholders.

1.2 Document Conventions

UML notation is used for diagrams (use case, sequence, state chart). Dates are written in ISO format YYYY-MM-DD. Requirements are numbered FR1, FR2... for functional and NFR1, NFR2... for non-functional requirements.

1.3 Intended Audience and Reading Suggestions

- Developers – focus on sections 2, 3 and 4.
- Testers – focus on functional requirements in section 4 and performance/security in section 5.
- Administrators – review product scope and business rules.

1.4 Product Scope

The RTVCS is a web-based application that allows multiple users to connect by video, share screens, chat by text and manage virtual rooms without installing extra plugins. It is available 24/7 through a web browser.

1.5 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- WebRTC specification (W3C) for real-time audio/video communication.

2. Overall Description

2.1 Product Perspective

The RTVCS is an independent, distributed internet application. Client browsers, webcams and microphones connect through a signaling server to start video calls.

2.2 Product Functions

- User registration and secure login.
- Instant creation of virtual rooms with custom Room IDs.
- Real-time video and audio communication.
- Text messaging inside the active room.
- Screen sharing and video layout control.

2.3 User Classes and Characteristics

- Meeting Host: creates the room and controls permissions.
- Participant: joins a room using a valid Room ID.
- System Administrator: manages server uptime and user accounts.

2.4 Operating Environment

- Hardware: laptop, desktop or mobile phone with webcam, microphone, speaker and internet connection.

- Software: modern web browser (Chrome, Firefox, Edge, Safari) with WebRTC support; server with signaling and media services.

2.5 Design and Implementation Constraints

- Must use WebRTC for peer-to-peer audio/video streaming.
- All media must be encrypted using SRTP.
- Must work on common browsers without installing plugins.

2.6 User Documentation

- On-screen help and tooltips.
- Online user guide for hosts and participants.

2.7 Assumptions and Dependencies

- Users have a working internet connection and a webcam/microphone.
- The browser allows camera, microphone and screen-capture permissions.
- STUN/TURN servers are available for connecting users behind firewalls.

3. External Interface Requirements

3.1 User Interfaces

- Simple, clean and responsive design for mobile phones and laptops.
- Buttons for mute/unmute, camera on/off, share screen, chat and leave meeting.

3.2 Hardware Interfaces

- Webcam, microphone, speakers/headphones and display screen.

3.3 Software Interfaces

- WebRTC API of the browser for audio/video streams.
- Database for users, rooms and sessions.
- Encryption library for secure data transmission.

3.4 Communications Interfaces

- HTTPS/WSS between browser and signaling server.
- SRTP for audio/video streams; UDP with STUN/TURN for NAT traversal.

4. System Features

4.1 System Feature 1 – Create Meeting Room

4.1.1 Description and Priority: Host creates a new room and gets a Room ID. High priority.

4.1.2 Stimulus/Response Sequences: Host clicks "New Meeting" – system generates a unique Room ID and opens the room.

4.1.3 Functional Requirements:

- FR1: The system shall allow only logged-in users to create a room.
- FR2: The system shall generate a unique Room ID for every room.

4.2 System Feature 2 – Join Meeting Room

Participant enters a valid Room ID and joins the meeting; the system shows an error for an invalid or expired Room ID.

- FR3: The system shall validate the Room ID before allowing entry.
- FR4: The system shall show the new participant's video to all users.

4.3 System Feature 3 – Audio/Video Control

User can mute/unmute the microphone and turn the camera on/off; others are notified of the change.

- FR5: The system shall stop or start audio/video tracks immediately on user request.

4.4 System Feature 4 – Text Chat

User sends text messages that are shown to all users in the room.

- FR6: The system shall deliver chat messages to all participants in real time.

4.5 System Feature 5 – Screen Sharing

User shares the desktop screen, which replaces the webcam view for all participants.

- FR7: The system shall ask for screen-capture permission before sharing.

4.6 System Feature 6 – Host Controls

Host can mute or remove a participant and end the meeting for everyone.

- FR8: The system shall allow only the host to remove participants or end the meeting.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- NFR1: Audio and video delay shall stay below 200 milliseconds on a normal network.
- NFR2: The system shall support at least 50 participants in one room.

5.2 Safety Requirements

- If a user's connection drops, the system shall remove the user from the room safely and notify others.

5.3 Security Requirements

- NFR3: All media streams shall be encrypted using SRTP.
- NFR4: Only users with a valid Room ID shall be able to join a room.
- NFR5: Passwords shall be stored in encrypted form.

5.4 Software Quality Attributes

- Reliability: 99.9% uptime.
- Usability: simple interface that works on mobile and desktop.
- Portability: runs on all major browsers.

5.5 Business Rules

- Only registered users can create meetings.
- The host has the right to remove any participant.

6. Other Requirements

Meeting session logs (join time, leave time) shall be kept in the database for administration.

Appendix A: Glossary

- SRS – Software Requirements Specification.
- IEEE – Institute of Electrical and Electronics Engineers.
- WebRTC – Web Real-Time Communication, used for peer-to-peer audio/video streaming.
- SRTP – Secure Real-time Transport Protocol.
- STUN/TURN – servers that help users connect through firewalls and NAT.
- Room ID – a unique code used to join a meeting room.

Appendix B: Analysis Models

The analysis models of the system (Use Case, Activity, Class, Sequence, Collaboration, State Chart, Component, Deployment and ER diagrams) are given in Experiments 2 to 10 and Experiment 12 of this manual.

Appendix C: To Be Determined List

None. All requirements of this version are fully specified.

Calculations:

Input: Functional and non-functional requirements of the video conferencing system
Output: Structured SRS document

Result: An IEEE-compliant Software Requirements Specification document for the Real-Time Video Conferencing System was created, outlining all major components of system behavior.

Precautions and Errors:

- Do not miss any required section.
- Keep requirements precise, measurable and in standard format.

Experiment No. 2

Aim: Draw the use case diagram and specify the role of each of the actors.

Learning Outcomes:

- Students will identify actors and use cases in a system.
- Students will understand how to represent user–system interaction through UML use case diagrams.
- Develop skills to capture system behaviour from the end-user's perspective.

Apparatus Used:

- Computer system
- UML modeling software (e.g., StarUML, Lucidchart, Draw.io)
- Text editor or diagramming tool

Main Constituents:

- Actors: Represent users or external systems interacting with the system.
- Use Cases: Define the functionalities provided by the system.
- System Boundary: Shows the limits of the system.
- Relationships: Associations between actors and use cases, and among use cases (include, extend).

Method / Procedure:

1. Identify the primary and secondary actors involved.
2. Identify the main functionalities of the system (use cases).
3. Establish the relationships between actors and use cases.
4. Draw the use case diagram using appropriate UML notations.
5. Validate the diagram with respect to the functional requirements.

Observation Table:

Actors:

- Meeting Host: Creates the meeting room, shares the Room ID, mutes or removes participants and ends the meeting.
- Participant: Joins a room using a valid Room ID, controls own camera and microphone, sends chat messages and shares the screen.
- System Administrator: Monitors connection channels and network speed, bans malicious users and maintains server configuration.

Use Cases:

- Register / Login
- Create Meeting Room
- Join Meeting Room
- Validate Room ID
- Toggle Media (Mute/Unmute)
- Text Chat
- Share Desktop Screen
- Mute / Remove Participant
- End Meeting
- Monitor Network
- Ban Malicious User

Relationships:

- Host → Register/Login, Create Meeting Room, Mute/Remove Participant, End Meeting.
- Participant → Register/Login, Join Meeting Room, Toggle Media, Text Chat, Share Desktop Screen.
- System Administrator → Monitor Network, Ban Malicious User.
- Join Meeting Room «include» Validate Room ID; Share Desktop Screen «extend» Toggle Media.

Calculations:

Input: User actions (e.g., login, join room, share screen)
Output: Use Case Diagram displaying actor-function relationships

Result: A use case diagram of the Real-Time Video Conferencing System was successfully created showing actors and their interaction with the system.

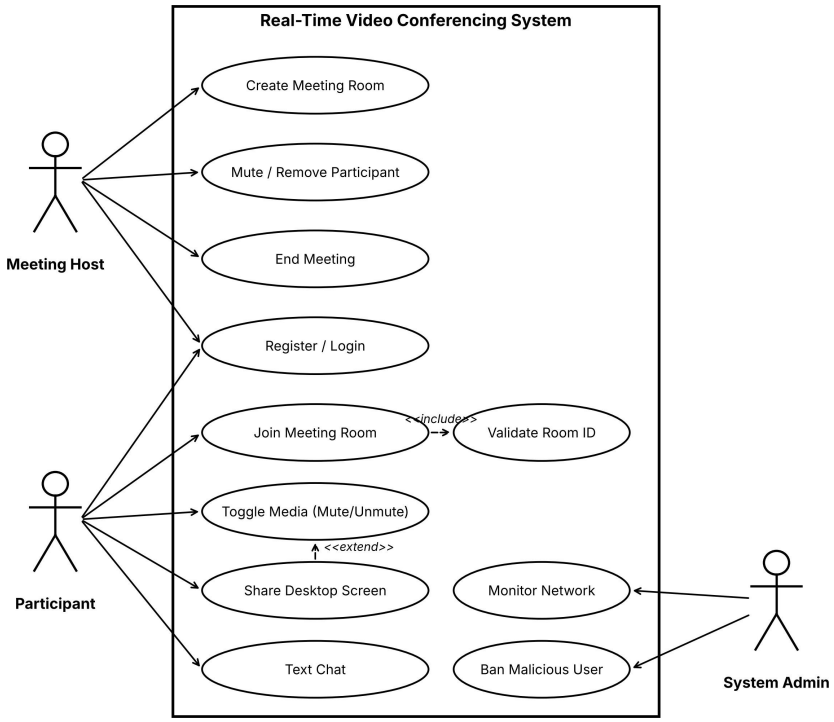


Figure 2.1: Use Case Diagram for Video Conferencing System

Precautions and Errors:

- Do not confuse roles with actors.
- Clearly separate system functions from user operations.
- Ensure consistency between use case names and requirements.

Experiment No. 3

Aim: Prepare state the precondition, post condition and function of each use case.

Learning Outcomes:

- Students will define system states before and after execution of use cases.
- Students will gain the ability to document functions, flows, and alternate scenarios effectively.
- Improve clarity in requirement documentation for developers and testers.

Apparatus Used:

- Computer system
- Text editor or UML tool
- Use case documentation template

Method / Procedure:

1. Identify the use cases defined in the system.
2. For each use case, document the preconditions, postconditions, and function.
3. Define the normal and alternative flow of events.
4. Review the documented use case with stakeholders to ensure clarity and correctness.

Observation Table:

Use Case 1: Create Meeting Room

- Function: Generates a new virtual meeting room and creates a shareable link.
- Precondition: The Host must be successfully logged in and connected to the internet.
- Postcondition: A unique Room ID is created on the server, and the host enters an empty video view layout.

Use Case 2: Join Meeting Room

- Function: Connects a participant to an ongoing call using an ID string.
- Precondition: The meeting room must already be started by a host, and the participant must input a matching, active Room ID.
- Postcondition: A peer-to-peer connection is set up, and the new user's video feed appears on all other screens.

Use Case 3: Toggle Media Stream (Mute/Unmute)

- Function: Stops or starts broadcasting audio or video tracks instantly.
- Precondition: The user is connected inside an active meeting room session.
- Postcondition: The local webcam or mic status changes to off or on, and a notification is sent to everyone else.

Use Case 4: Text Chat Interaction

- Function: Sends instantaneous text strings to all users inside the active room.
- Precondition: The user is inside an active room session.
- Postcondition: The text message is broadcasted through the data channel and shows up on everyone's chat sidebar.

Use Case 5: Share Desktop Screen

- Function: Captures the user's desktop window and streams it instead of the webcam feed.
- Precondition: The user is in an active call and the browser has screen capture permission.
- Postcondition: The user's webcam stream is temporarily replaced by the screen share for all participants.

Flow of Events (Join Meeting Room):

1. Participant opens the application and enters the Room ID.

2. Client sends a join request to the signaling server.
3. System verifies the Room ID with the database.
4. If the Room ID is invalid, display an error message and return to step 1.
5. If the Room ID is valid, set up the peer-to-peer connection.
6. Notify the host that a new participant has joined.
7. Display the video grid to the participant.

Calculations:

Input: Room ID, user login session, camera and microphone permission
Output: Participant joined in the meeting room with video and audio

Result: Preconditions, postconditions, function and flow of events for the use cases were successfully documented.

Precautions and Errors:

- Do not skip alternative flows or exceptional scenarios.
- Ensure logical consistency in steps.
- Preconditions must be verifiable.

Experiment No. 4

Aim: Draw the activity diagram.

Learning Outcomes:

- Represent workflows and system processes using UML activity diagrams.
- Students will understand decision points, concurrency, and flow of control in a system.
- Learn how to visualize business logic and system workflows step by step.

Apparatus Used:

- Computer system
- UML tool (e.g., draw.io, Lucidchart, StarUML)
- Document editor

Basic Components:

- Action Box: Represents a task or activity.
- Initial State: Starting point of the process.
- Final State: Ending point of the process.
- Decision Node: Represents a conditional path.
- Control Flows: Show transition between activities.

Procedure:

1. Identify major processes within the system.
2. For each process, determine the flow and alternatives.
3. Use standard UML symbols (initial state, action, decision, final state).
4. Draw transitions between actions and decision points.
5. Verify completeness and correctness of the activity flow.

Observation Table:

Example Activity Diagram: Join Meeting Process

- Initial State: Start of session.
- Actions: Login, Enter Room ID, Send join request, Set up peer connection, Start audio/video, Chat / share screen, Leave meeting.
- Decision Box: Is the Room ID valid? Is media permission granted?
- Final State: End of the meeting session.

Calculations:

Input: Use case scenario – join meeting
Output: Activity diagram representing the system workflow

Result: The activity diagram was successfully created to represent the flow of control for system activities and decisions.

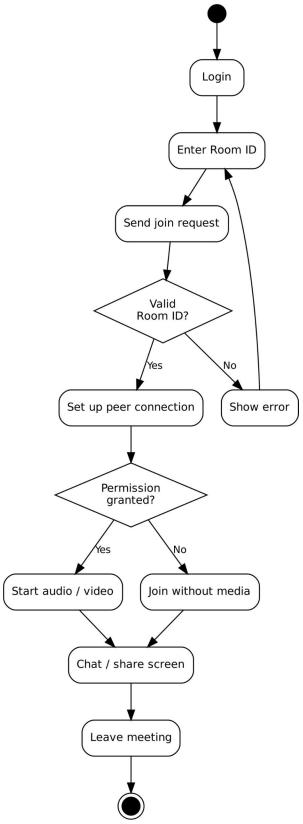


Figure 4.1: Activity Diagram – Join Meeting

Precautions and Errors:

- Ensure arrows flow logically between states.
- Include all alternative/decision branches.
- Avoid cyclic or ambiguous flows.

Experiment No. 5

Aim: Identify the classes. Classify them as weak and strong classes and draw the class diagram.

Learning Outcomes:

- Students will identify system classes, attributes, and methods.
- Students will understand how to differentiate between strong and weak classes and model their relationships.
- Gain skills to design the backbone of object-oriented software systems.

Apparatus Used:

- Computer system
- UML design tool (draw.io, StarUML, Lucidchart, etc.)
- Document editor (MS Word, LibreOffice Writer, etc.)

Procedure:

1. Analyze the system or case scenario.
2. Identify the classes and their attributes/methods.
3. Classify classes as strong or weak based on independence.
4. Define relationships among classes (inheritance, aggregation, etc.).
5. Draw the class diagram using standard notations.

Observation Table:

Class	Type	Attributes / Methods
User	Strong	userId, name, email, password / register(), login()
Host	Strong	createRoom(), muteUser(), removeUser(), endMeeting() (inherits User)
Participant	Strong	joinRoom(), toggleMedia(), shareScreen() (inherits User)
Admin	Strong	monitorServer(), banUser() (inherits User)
Room	Strong	roomId, createdAt, status / start(), end()
Message	Weak	text, timestamp / send() – depends on Room
MediaStream	Weak	type, status / start(), stop() – depends on Room

Relationships: Inheritance from User to Host, Participant and Admin; Association between Host/Participant and Room; Composition between Room and Message / MediaStream.

Result: Class diagram was drawn successfully and classes were correctly identified as weak or strong based on their dependencies.

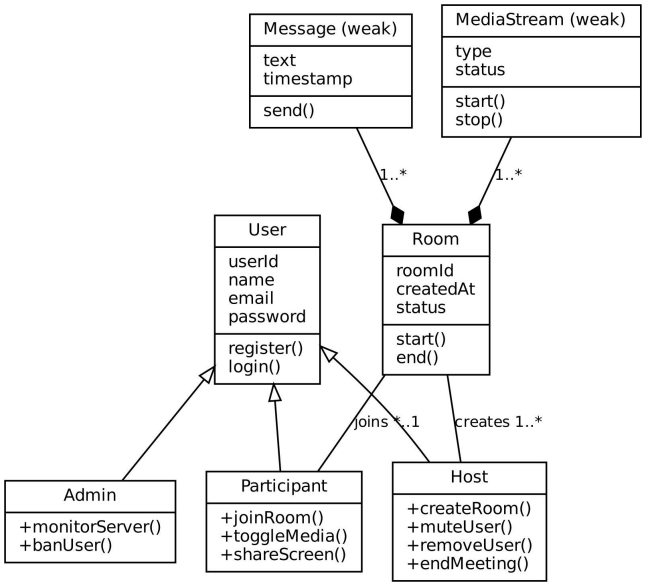


Figure 5.1: Class Diagram of Video Conferencing System

Precautions and Errors:

- Ensure correct identification of relationships (inheritance, association).
- Include all attributes and operations relevant to the system.
- Maintain consistent UML syntax.

Experiment No. 6

Aim: Draw the Sequence Diagram for Any Two Scenarios

Learning Outcomes:

- Students will create models of step-by-step object interactions for different scenarios.
- Students will gain knowledge of message passing, lifelines, and event sequences.
- Understand how to represent the order and timing of operations within a system.

Apparatus Used:

- Computer system with UML design tools (e.g., draw.io, StarUML, Creately, Lucidchart)
- Microsoft Word or similar software for documentation

Basic Sequence Diagram Elements:

- Actors: External entities that interact with the system.
- Objects (Lifelines): Vertical dashed lines that represent the existence of an object over a period of time.
- Messages: Horizontal arrows between lifelines representing communication.
- Activation Bars: Represent the time an object is performing an operation.

Procedure:

1. Identify the scenario for which the sequence diagram needs to be drawn.
2. Identify all participating objects and actors.
3. Define the sequence of interactions and messages.
4. Draw the lifelines for each object.
5. Draw arrows to represent messages exchanged between objects.
6. Label each message appropriately.
7. Add activation bars to show processing.

Observation Table (Program Prints or Diagrams):

Scenario 1: Join Meeting Room
Participants: Participant, Web Client, Signaling Server, Database, Host

Participant -> Web Client : enter Room ID
Web Client -> Signaling Server : joinRequest(roomId)
Signaling Server -> Database : validateRoom(roomId)
Database -> Signaling Server : room active
Signaling Server -> Host : notify new participant
Signaling Server -> Web Client : offer / ICE candidates
Web Client -> Signaling Server : answer / ICE candidates
Web Client -> Participant : video grid displayed

Scenario 2: Share Desktop Screen
Participants: Participant, Web Client, Browser API, Media Server, Other Users

Participant -> Web Client : click Share Screen
Web Client -> Browser API : getDisplayMedia()
Browser API -> Participant : permission prompt
Participant -> Browser API : allow
Browser API -> Web Client : screen stream
Web Client -> Media Server : replace video track
Media Server -> Other Users : forward screen stream

Other Users -> Participant : screen view confirmed

Calculations / Sample Input & Output:

Not applicable for this experiment.

Result: The sequence diagrams for both the Join Meeting Room scenario and the Share Desktop Screen scenario were successfully created, showing the flow of messages and interactions.

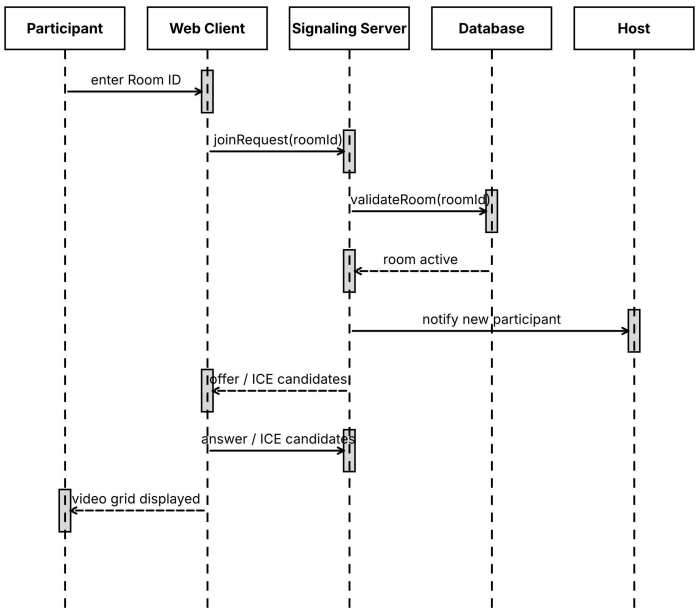


Figure 6.1: Join Meeting Room Sequence Diagram

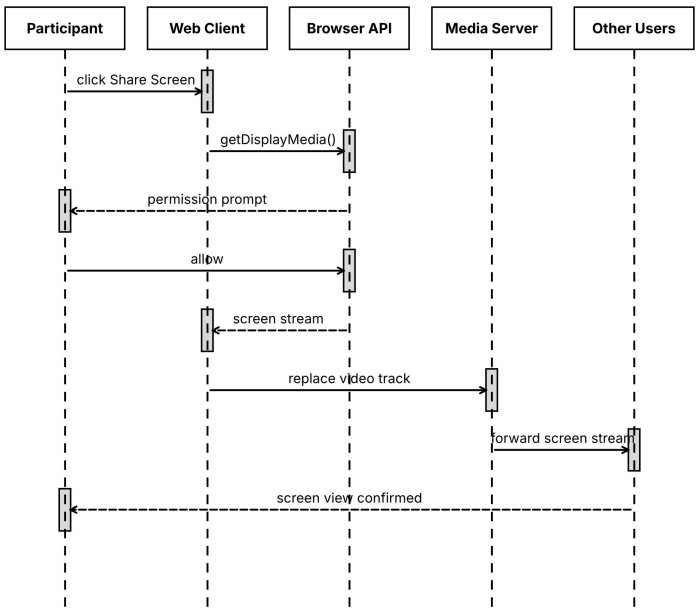


Figure 6.2: Share Desktop Screen Sequence Diagram

Precautions and Errors:

- Ensure correct order of interactions.
- Clearly identify all objects and actors.
- Avoid crossing message arrows for readability.
- Maintain correct UML notation.

Experiment No. 7

Aim: Draw the Collaboration Diagram

Learning Outcomes:

- Represent relationships and communication among objects.
- Students will understand how to highlight structural links and the order of messages exchanged.
- Learn to analyse system structure along with behaviour in a single diagram.

Apparatus Used:

draw.io, Microsoft Visio, StarUML, Lucidchart, Creately or any UML Drawing Tool.

Procedure:

1. Identify the participating objects in a use case.
2. Determine the sequence of interactions.
3. Label each interaction as a message between objects.
4. Connect the objects using links that represent their relationships.
5. Use arrows to indicate the direction and order of message passing.

Observation Table:

The following diagram illustrates a collaboration between a Participant, Web Client, Signaling Server, Database and Media Server for joining a meeting room:

- 1: enterRoomId() – Participant to Web Client
- 2: joinRequest() – Web Client to Signaling Server
- 3: validateRoom() – Signaling Server to Database
- 4: roomActive() – Database to Signaling Server
- 5: connect() – Web Client to Media Server
- 6: offer / answer – between Web Client and Signaling Server
- 7: sendStream() – Web Client to Media Server
- 8: showVideo() – Web Client to Participant

Calculations:

Sample input: Room ID entered and Join clicked
Expected Output: User joined successfully and video of the meeting is displayed

Result: The collaboration diagram was successfully drawn and the interaction among the objects was clearly represented.

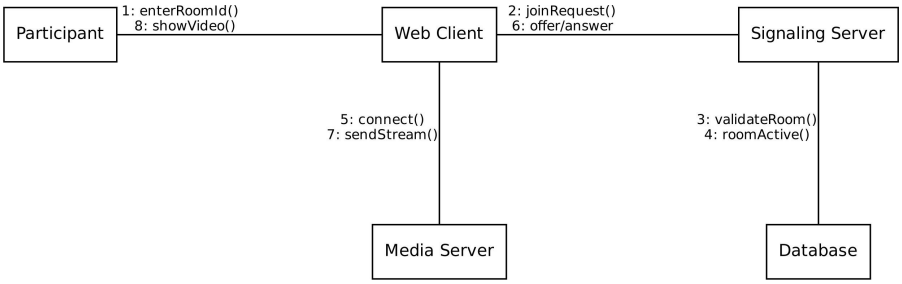


Figure 7.1: Collaboration Diagram of Joining a Meeting Room

Precautions and Errors:

- Ensure correct labeling and order of messages.
- Avoid circular references unless they represent a valid interaction loop.

Experiment No. 8

Aim: Draw the State Chart Diagram

Learning Outcomes:

- Model object states and transitions based on events.
- Students will understand event-driven behaviour in reactive systems.
- Gain the ability to represent real-world scenarios such as ATM or online transaction workflows.

Apparatus Used:

- Computer System
- UML Modelling Tool (e.g., draw.io, StarUML, Creately, Visual Paradigm)
- MS Word or PDF viewer

Key Notations:

- Initial State: Represented by a filled black circle.
- Final State: Represented by a circle with a dot inside.
- State: Rounded rectangles depicting conditions.
- Transition: Arrows between states with event labels.
- Event: Trigger that causes a transition between states.

Method / Procedure (Algorithm to Construct a State Diagram):

1. Identify the object to be modeled (e.g., a participant in a meeting).
2. List all possible states (e.g., Idle, Joining, In Call, etc.).
3. Determine events that trigger transitions (e.g., valid Room ID, mute, leave).
4. Draw states and transitions using appropriate UML notations.
5. Validate all paths from start to end states.

Observation Table (State Chart Diagram):

The participant goes through several states from the moment the application is opened until the meeting is left.
Explanation of States:

- Idle – Application is open but user is not in a meeting.
- Joining – Room ID is entered and being checked.
- Connected – Room ID is valid and connection is established.
- In Call – Audio and video are running.
- Muted – Microphone or camera is turned off.
- Sharing – Screen sharing is active.
- Left – User has left or the host has ended the meeting (final state).

Calculations (Sample Input & Output):

Input	**Output**
Valid Room ID entered	Connection established
Mute pressed	Audio stopped, others notified
Share Screen pressed	Screen shown to all participants
Leave pressed anytime	User disconnected from the meeting

Result: The state chart diagram of the participant was successfully designed, illustrating all possible transitions and final states of user interaction.

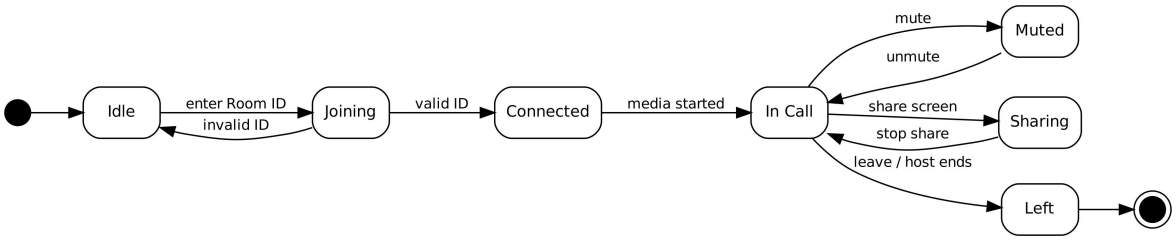


Figure 8.1: State Transition Diagram of a Participant

Precautions and Errors:

- Ensure all possible events and error cases are captured.
- Each state must have at least one outgoing transition unless it's a final state.
- Avoid ambiguous transitions.

Experiment No. 9

Aim: Draw the Component Diagram.

Learning Outcomes:

- Students will understand how to visualize the modular structure of the software system.
- Students will understand provided and required interfaces between components.
- Learn how to model reusable and replaceable parts of a large system.

Apparatus Used: draw.io, Microsoft Visio, StarUML, MS PowerPoint

Procedure:

1. Identify all the components in the system such as Web Client, Signaling Server, Media Server etc.
2. Determine interfaces and dependencies among the components.
3. Create the component boxes with required notations.
4. Connect components using dependency arrows to show the flow and interaction.
5. Use symbols like provided and required interfaces as applicable.
6. Validate the connections and interaction consistency.

Observation Table:

Component	**Description**
Web Client	Browser application used by the user
Camera & Mic	Hardware used to capture audio and video
Signaling Server	Sets up sessions between users
Authentication	Handles registration and login
Room Manager	Creates and manages meeting rooms
Media Server	Routes audio and video streams
STUN/TURN Server	Helps users connect through firewalls
Database	Stores users, rooms and sessions

Result: The component diagram successfully models the dependencies and interactions between various components of the video conferencing system.

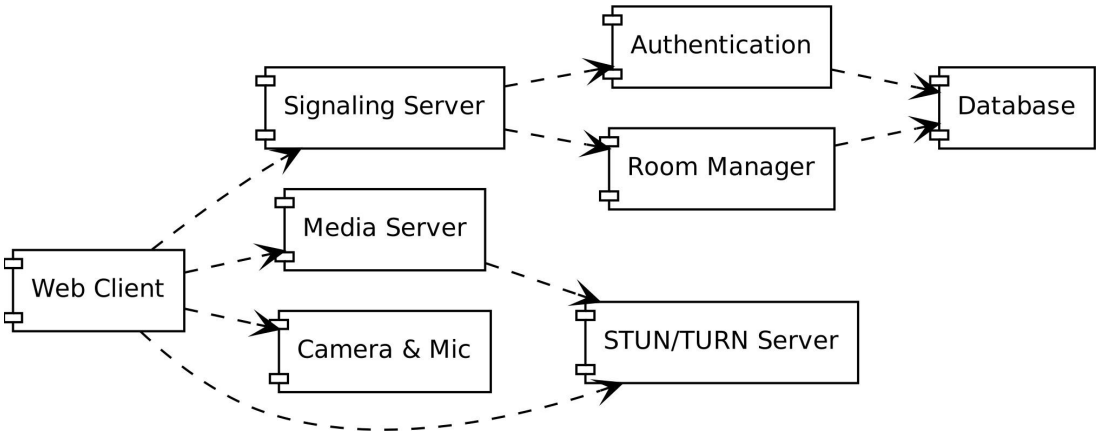


Figure 9.1: Component Diagram of Video Conferencing System

Precautions and Errors:

- Ensure proper usage of provided and required interfaces.
- Maintain correct directional flow of dependencies.
- Avoid clutter and label each component clearly.

Experiment No. 10

Aim: Draw the Deployment Diagram

Learning Outcomes:

- Students will represent physical deployment of software artifacts on hardware nodes.
- Students will gain knowledge of runtime architecture in distributed systems.
- Understand system execution environments and communication paths.

Apparatus Used: draw.io, Computer system, StarUML or any UML tool, Microsoft PowerPoint.

How to develop Deployment Diagrams:

1. Decide on the purpose of the diagram.
2. Add nodes to the diagram.
3. Add communication associations to the diagram.
4. Add other elements to the diagram, such as components or active objects, if required.
5. Add dependencies between components and objects, if required.

Procedure:

Step 1: Open a UML diagramming tool.
Step 2: Create nodes representing hardware elements.
Step 3: Add components and artifacts within those nodes.
Step 4: Define communication paths using dependency arrows.
Step 5: Label each part of the system clearly.

Observation Table:

Node	**Software Deployed**	**Connected To (Protocol)**
User Laptop / Phone	Web Browser	Signaling Server (HTTPS/WSS), Media Server (SRTP), STUN/TURN (UDP)
Signaling Server	WebSocket service	Database Server (TCP/IP)
Media Server (SFU)	Audio/video routing	User devices (SRTP)
STUN/TURN Server	NAT traversal service	User devices (UDP)
Database Server	Users, rooms, sessions	Signaling Server (TCP/IP)

Result: A deployment diagram for the Real-Time Video Conferencing System has been successfully drawn showing the distribution of software components and their hosting environment.

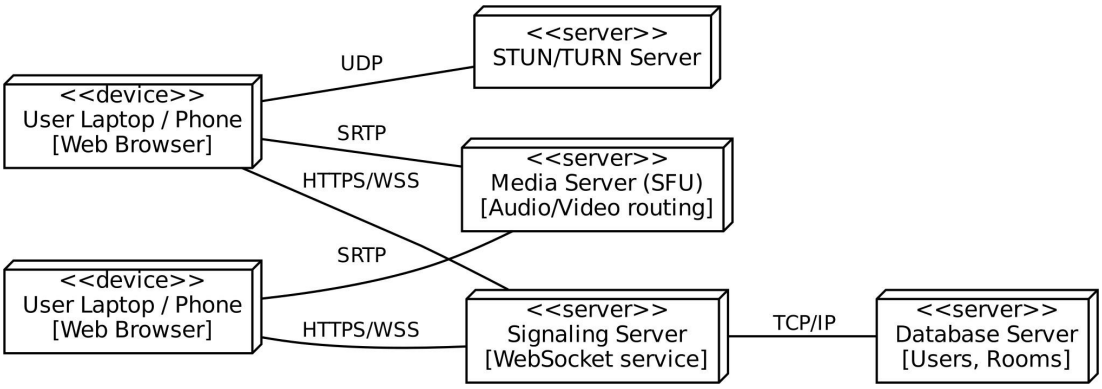


Figure 10.1: Deployment Diagram

Precautions and Errors:

- Ensure all components and nodes are labeled.
- Maintain correct associations between nodes and components.
- Use correct UML notation.

Experiment No. 11

Aim: Test Case for Video Conferencing System Functionality

Learning Outcomes:

- Students will understand the process of systematic test case design and execution.
- Validate functional requirements through real-world application testing.
- Gain practical exposure to quality assurance and error detection techniques.

Apparatus Used: Computer System, Web Browser, Webcam and Microphone, Internet Connection, RTVCS user account

Procedure:

1. Identify key features of the RTVCS: Login, Meeting Room, Audio/Video control, Chat and Screen Sharing.
2. Document test cases in a descriptive format.
3. Execute each test case.
4. Record the actual results and validate against expected outcomes.

Observation Table:

Test Cases for Meeting Room Functionality:

- Verify that on clicking 'New Meeting', a unique Room ID is generated.
- Verify that the Room ID can be copied and shared with other users.
- Verify that a user can join a meeting by entering a valid Room ID.
- Verify that an error message is shown for an invalid or expired Room ID.
- Verify that the video of a newly joined participant is displayed to all users.
- Verify that the participant count is updated when a user joins or leaves.
- Verify that the host can mute or remove a participant.
- Verify that when the host ends the meeting, all participants are disconnected.

Test Cases for Audio, Video, Chat and Screen Sharing:

- Verify that clicking the mic button mutes or unmutes the microphone and others see the status.
- Verify that clicking the camera button turns the video on or off.
- Verify that the browser asks permission for camera and microphone before the call starts.
- Verify that a chat message typed by a user appears for all participants.
- Verify that chat messages are shown in the correct order with sender name and time.
- Verify that clicking 'Share Screen' asks for permission and shows the screen to all participants.
- Verify that stopping the screen share brings back the webcam video.
- Verify that audio and video delay stays below 200 milliseconds on a normal network.

Test Case ID	Test Case Description	Pre-Condition	Test Steps	Expected Result
TC_LOGIN_001	Verify login with valid credentials	User account exists	1. Open login page 2. Enter valid email and password 3. Click 'Login'	User is logged in and dashboard is displayed
TC_LOGIN_002	Verify login with invalid password	User account exists	1. Open login page 2. Enter valid email, wrong password 3. Click 'Login'	Error message: 'Wrong password'

Test Case ID	Test Case Description	Pre-Condition	Test Steps	Expected Result
TC_LOGIN_003	Verify login with non-registered email	None	1. Open login page 2. Enter unregistered email 3. Click 'Login'	Error: 'Account not found'
TC_ROOM_001	Verify creating a meeting room	User is logged in	1. Click 'New Meeting'	Unique Room ID is generated and room opens
TC_ROOM_002	Verify joining with valid Room ID	Room is active	1. Enter Room ID 2. Click 'Join'	User joins and video appears for all
TC_ROOM_003	Verify joining with invalid Room ID	None	1. Enter wrong Room ID 2. Click 'Join'	Error: 'Room not found'
TC_MEDIA_001	Verify muting the microphone	User is in a meeting	1. Click mic button	Audio stops; others see muted icon
TC_MEDIA_002	Verify turning off the camera	User is in a meeting	1. Click camera button	Video stops; others see a blank tile
TC_CHAT_001	Verify sending a chat message	User is in a meeting	1. Type text 2. Press Send	Message appears for all participants
TC_SCREEN_001	Verify screen sharing	User is in a meeting	1. Click 'Share Screen' 2. Allow permission	Screen is shown to all participants
TC_HOST_001	Verify host removing a participant	Host and participant are in a meeting	1. Host selects 'Remove' on the participant	Participant is disconnected from the room
TC_HOST_002	Verify host ending the meeting	Host is in a meeting	1. Host clicks 'End Meeting'	All participants are disconnected
TC_LOGOUT_001	Verify logout	User is logged in	1. Click profile icon 2. Select 'Sign out'	User is logged out and login page is displayed

Result: All the test cases were successfully documented for the Meeting Room, Audio/Video, Chat and Screen Sharing functionalities of the Real-Time Video Conferencing System.

Precautions and Errors:

- Ensure stable internet connection while using the application.
- Allow camera, microphone and screen-capture permissions in the browser.
- Use valid and testable user accounts for testing.

Experiment No. 12

Aim: Understanding and Drawing an ER Diagram

Learning Outcomes:

- Students will understand database design concepts using ER modelling.
- Students will gain the ability to represent entities, attributes, relationships, and constraints.
- Develop the foundation for implementing efficient relational databases.

Apparatus Used:

- ER diagram tool (e.g., draw.io or Lucidchart)
- Problem domain description (Real-Time Video Conferencing System with users, rooms and messages)

Procedure:

1. Identify relevant entities (e.g., User, Room, Message).
2. List attributes: User (userId, name, email), Room (roomId, createdAt, hostId), Message (msgId, text, time).
3. Define relationships (e.g., User hosts Room).
4. Assign cardinalities and participation: e.g., one user can host many rooms (1–M).
5. Use tool to draw boxes, name them, connect with lines and diamond shapes, label constraints.

Observation Table:

Entity	Attributes	Keys
User	userId, name, email	userId
Room	roomId, createdAt, hostId	roomId (PK), hostId (FK)
Message	msgId, text, time	msgId
Participation	joinTime, leaveTime	composite userId + roomId

Calculations:

Not applicable.

Result: Completed ER diagram accurately representing the video conferencing database, including correct symbols and constraints.

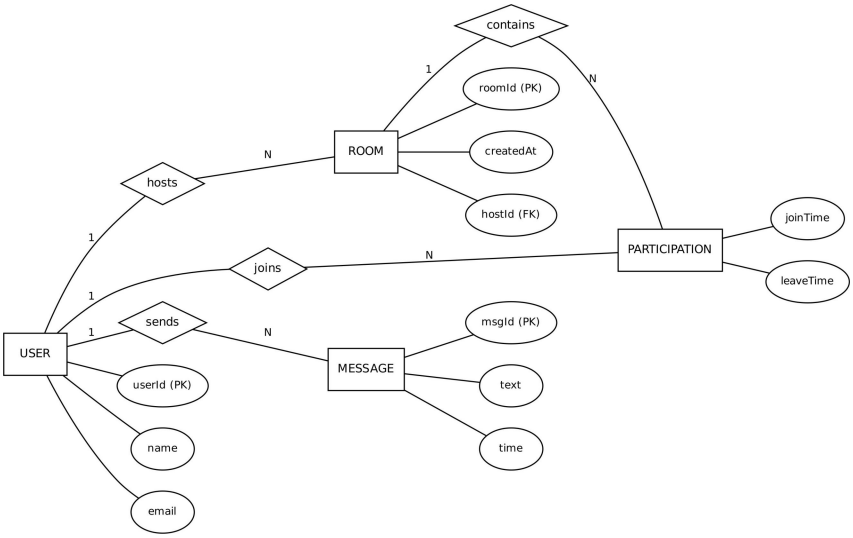


Figure 12.1: ER Diagram for Video Conferencing System

Precautions and Errors:

- Ensure every entity has a key attribute.
- Use correct symbols consistently.
- Clarify cardinality (e.g., whether many rooms per user and vice versa).
- Prevent weak or ambiguous relationships.